

EXPLICACION

La pila se utilizó como **estructura de apoyo** para convertir expresiones de notación infija a posfija.

- Cada vez que aparece un **operador** o un **paréntesis**, se guarda en la pila.
- Cuando se encuentra un **operando**, se envía directamente a la salida.
- Al cerrar un paréntesis o al comparar prioridades, se desapilan operadores y se agregan a la salida en el orden correcto.
- Al final, se vacía la pila y se completa la expresión posfija.

☞ En resumen: la pila garantiza que los operadores y paréntesis se procesen en el orden adecuado, respetando la jerarquía de operaciones.

Código limpio e integrado:

```
from estructuras.lineales.stack import OperacionesPila
from estructuras.lineales.lista_enlazada_simple import LinkedList
```

```
class ConvertidorExpresiones:
```

```
    def __init__(self):
```

```
        # La pila se implementa sobre una lista enlazada
```

```
        self.pila = OperacionesPila(LinkedList())
```

```
    def prioridad(self, operador):
```

```
        if operador == '$': # Potencia
```

```
            return 3
```

```
        if operador in ('*', '/')
```

```
            return 2
```

```
        if operador in ('+', '-'):
```

```
            return 1
```

```
        return 0
```

```
    def infija_a_posfija(self, expresion):
```

```
        salida = []
```

EXPLICACION

for token in expresion:

if token.isalnum(): # Operando

salida.append(token)

elif token == '(':

self.pila.pila_agregar(token)

elif token == ')':

while not self.pila.is_empty() and self.pila.pila_tope() != '(':

salida.append(self.pila.pila_eliminar())

self.pila.pila_eliminar() # eliminar '('

else: # Operador

while (not self.pila.is_empty() and

self.prioridad(self.pila.pila_tope()) >= self.prioridad(token)):

salida.append(self.pila.pila_eliminar())

self.pila.pila_agregar(token)

Vaciar pila al final

while not self.pila.is_empty():

salida.append(self.pila.pila_eliminar())

return " ".join(salida)